

Guía de Práctica Parcial Concurrente I

Cómo deducir cada tipo de ejercicio + respuestas tipo

Basado en parciales 1c2025, 1c2025-recu, 2c2024, 1c2024, 2c2023

#	Tema
1	Modelo de concurrencia — cómo elegirlo según el enunciado
2	Verdadero / Falso — patrones y respuestas tipo
3	¿Es busy wait? — checklist de decisión
4	Corrección de código — errores típicos
5	Modelar con Actores — receta + ejemplos resueltos
6	Redes de Petri — clásicos del parcial
7	Resumen final — chuleta de 1 página

1. Modelo de Concurrency — cómo elegirlo

Estrategia de 3 pasos

- **Paso 1:** ¿Qué consume principalmente la tarea? CPU, I/O, o estado reactivo.
- **Paso 2:** ¿Hay estado compartido entre subtareas? Si no comparten → Fork-Join. Si comparten y todos modifican → Mutex. Si reaccionan a eventos → Actores.
- **Paso 3:** Justificar mencionando por qué los otros modelos no sirven.

Tabla de decisión rápida

Pista en el enunciado	Modelo	Por qué
Procesar/transformar muchos archivos, frames, imágenes, registros	Fork-Join	CPU-bound, datos independientes, paralelizable, determinístico
Multiplicar matrices, entrenar red neuronal, renderizar 3D	Fork-Join	Cómputo intensivo puro, paralelismo de datos
Encriptar/comprimir N archivos con clave compartida solo de lectura	Fork-Join	Solo lectura compartida → no requiere sincronización
Consultar APIs externas, scraping, descargar de internet	Async	I/O-bound, alta latencia de red, await libera el thread
Servidor que maneja miles de conexiones HTTP concurrentes	Async	I/O pura, máxima concurrencia con bajo overhead
Cache compartida con muchos lectores y pocos escritores	Estado mutable compartido (RwLock)	Lectura concurrente, escritura serializada
Carrito/cuenta/contador compartido modificado por muchos	Estado mutable compartido (Mutex)	Escrituras serializadas, exclusión mutua
Subasta en tiempo real, chat, juego online, MMOG	Actores	Cada entidad reacciona a eventos, estado aislado, escalable
Backend de restaurante donde mozos cobran/anotan en mesas compartidas	Estado mutable compartido	Cuentas y pedidos son datos compartidos modificados simultáneamente
Sistema con recurso limitado a N usos simultáneos (rate limiting)	Async + Semáforo o Actores	Semáforo controla acceso al recurso, async para las llamadas

Cuándo NO es lo que parece

■ Async para cómputo intensivo → ¡MAL!

Las tareas async **no ceden la CPU** si no hay await. Para CPU-bound usar Fork-Join.

■ Fork-Join con estado compartido mutable → ¡MAL!

Fork-Join **asume independencia**. Si subtareas modifican estado común → race conditions.

■ Barriers + Mutex para llamadas a APIs → ¡MAL!

Las APIs son I/O-bound. Barriers obligan a **sincronizar todos** con la respuesta más lenta. Usar async.

■ Vectorización para sincronización → ¡MAL!

Vectorización es para acelerar **cálculos numéricos**, no para coordinar acceso a recursos.

Plantilla de respuesta

[NOMBRE DEL MODELO]. [CASO DE USO] es una tarea de [CPU-bound / I/O-bound / estado reactivo]. Este modelo es ideal porque [VENTAJA PRINCIPAL]. Permite [BENEFICIO CONCRETO: paralelizar / ceder control en await / aislar estado / serializar acceso]. Otros modelos como [OTRO] no convienen porque [POR QUÉ NO].

Ejemplos resueltos del práctica-parcial

Caso de uso	Modelo correcto
Encriptar múltiples PDFs con clave privada	Fork-Join
Backend de restaurante con mozos compartiendo mesas	Estado mutable compartido
Consultar cotizaciones de Bitcoin en cientos de exchanges	Async
Subtitular videos con red neuronal (voz a texto)	Fork-Join
Consultas en tiempo real sobre miles de tablas	Fork-Join
Backend de MMOG (juego masivo online)	Actores
Crawler que descarga internet	Async
Generar deepfakes a partir de prompts	Fork-Join
Subasta en línea con notificación a usuarios	Actores
Aplicar filtros de color a una imagen	Fork-Join (un pixel = una subtarea)
Cache de Machine Learning compartida	Estado mutable compartido (RwLock)
Convertir N grupos de 100 imágenes a PDF	Fork-Join
Llamadas a múltiples APIs y combinar resultados	Async
Procesar logs masivos de un sitio web concurrido	Async
Backend para Google Docs colaborativo	Actores
Convertir DOC a PDF en lote	Fork-Join
Backend de Kahoot/Menti (preguntas y respuestas)	Actores o canales-mensajes
API HTTP de NLP para clasificar sentimientos	Async

2. Verdadero / Falso — patrones y respuestas tipo

Estrategia general

■ Casi todas las afirmaciones del parcial son FALSAS.

Los profesores apuntan a desarmar conceptos confusos. Si dudás, primero buscá el truco.

- Buscar palabras absolutas: **siempre, solo, nunca, únicamente** → suelen ser exageraciones.
- Buscar mezclas conceptuales: actores con locks, async bloqueante, fork-join con races → falsas por diseño.
- Verificar definiciones: `poll()` no bloquea, `async fn` no ejecuta, threads comparten memoria con tasks.

Banco de afirmaciones — todas las del práctica-parcial

Afirmación	R	Justificación
Procesos, hilos y tareas asincrónicas tienen espacios de memoria independientes	F	Solo los procesos. Hilos y tasks comparten heap/global del proceso.
El scheduler del SO puede detener una tarea async y habilitar otra	F	El scheduler del SO solo gestiona threads. Las tasks usan modelo cooperativo (Executor) y solo ceden con <code>await</code> .
En vectorización las operaciones verticales y horizontales devuelven vectores	F	Verticales sí (lane a lane); horizontales reducen a un escalar.
Un hilo en <code>condvar</code> solo despierta cuando otro hace signal	F	Existen spurious wakeups. Por eso siempre <code>while</code> , no <code>if</code> .
Las tareas asincrónicas son ideales para cómputo intensivo	F	Async es para I/O-bound. CPU-intensivo → <code>fork-join</code> .
El estado interno de un actor puede accederse desde otros con locks	F	Solo se accede mediante mensajes. La aislación la garantiza el mailbox.
Programando en <code>fork-join</code> hay que cuidar las race conditions	F	<code>Fork-join</code> asume tareas independientes sin estado mutable compartido. Es determinístico.
En estado mutable compartido los hilos verifican periódicamente actualizaciones	F	Eso sería busy wait. Lo correcto es <code>Condvar::wait()</code> o Semáforos (espera pasiva).
En 1 CPU, hilos intensivos tardan menos que tareas async equivalentes	F	En 1 CPU solo hay concurrencia (intercalación). Threads tienen alto costo de context switch; tasks son más livianas.
El estado interno de un actor está protegido por exclusión mutua	F	Es privado por aislamiento, no por exclusión mutua. La serialización la da el mailbox internamente.
El encargado de hacer <code>poll</code> es el thread principal	F	Lo hace el Executor del runtime async (ej: Tokio), que puede correr en cualquier thread del runtime.

poll() es llamado solo cuando la función puede progresar	F	El Executor llama a poll cuando cree que un recurso podría estar listo. Puede retornar Pending sin progresar.
El modelo piñata es colaborativo	V	Las tareas ceden el control voluntariamente con await. El Executor solo cambia cuando la tarea se suspende.
La operación async inicia cuando se llama al método async	F	Retorna inmediatamente un Future. La ejecución real comienza cuando el Executor hace poll.
Los threads y las tasks tienen un stack propio	F	Solo los threads. Las tasks son máquinas de estado livianas, sin stack tradicional.
La llamada a poll de un Future es bloqueante	F	Nunca bloquea. Retorna Ready o Pending para que el Executor trabaje en otra cosa.

Cheatsheet de mitos a desarmar

- poll() NUNCA bloquea — siempre retorna Ready o Pending.
- async fn NO ejecuta al llamarla — retorna un Future.
- Spurious wakeups EXISTEN — siempre usar while, no if.
- Threads y tasks COMPARTEN memoria del proceso — solo procesos están aislados.
- Fork-Join NO tiene race conditions por diseño — las subtareas son independientes.
- Tasks NO tienen stack — son máquinas de estado.
- Actor NO usa Mutex para su estado — el aislamiento es por arquitectura.

3. ¿Es busy wait? — checklist de decisión

Definición

Busy wait (espera activa): un thread queda atrapado en un bucle de verificación rápida de una condición que no se resuelve, gastando ciclos de CPU sin hacer trabajo útil.

■ **Pregunta clave:** ¿el thread cede la CPU eficientemente mientras espera?

Cuándo NO es busy wait (espera pasiva correcta)

- **Bloqueo pasivo:** el código usa `lock()`, `acquire()`, `Condvar::wait()`, `rx.recv()` — el SO suspende el thread (consume 0 CPU).
- **Cesión explícita larga:** `thread::sleep(DELAY_LARGO)` después de hacer trabajo útil productivo.
- **expect() sobre lock:** `lock().expect("...")` es bloqueante, espera pasiva.
- **Filósofo bien implementado:** primer palillo con `acquire()` (bloqueante), segundo también con `acquire()` → espera pasiva.

Cuándo SÍ es busy wait (espera activa)

- **Bucle con verificación:** `loop { if condicion { break } } o while !lista { ... }` → gasta CPU preguntando.
- **Reintento rápido sin bloqueo:** `try_acquire()`, `try_lock()`, `try_recv()` en loop sin sleep largo.
- **Filósofo con try_acquire:** adquiere primer palillo bloqueante, segundo con `try_acquire` → si falla, libera y reintentará inmediatamente.
- **Sleep corto + try:** `sleep(DELAY_CORTO) + try_lock` sigue siendo busy wait disimulado (chequeo activo periódico).
- **TcpStream::connect en loop:** reintento de conexión en bucle es busy wait.
- **Falta de Condvar:** usar solo Mutex sin Condvar para esperar cambio de estado obliga a chequear en bucle.

Plantilla de decisión

¿Hay un loop o while?

■ NO → no es busy wait

■ SÍ → ¿qué hace dentro?

- | | |
|--|----------------|
| ■ <code>lock()</code> / <code>acquire()</code> / <code>wait()</code> / <code>recv()</code> | → NO busy wait |
| ■ <code>try_lock()</code> / <code>try_acquire()</code> / <code>is_some()</code> | → SÍ busy wait |
| ■ <code>TcpStream::connect()</code> en loop sin sleep | → SÍ busy wait |
| ■ <code>sleep(DELAY_LARGO) + trabajo útil</code> | → NO busy wait |
| ■ <code>sleep(DELAY_CORTO) + chequeo activo</code> | → SÍ busy wait |

Ejemplos resueltos del práctica-parcial

Ejemplo 1 — Mineros con `copper.write().expect()`

```
for _ in 0..MINERS {
  let copper = Arc::clone(&resource);
  thread::spawn(move || loop {
    let mined = rand::thread_rng().gen_range(1..10);
    *copper.write().expect("failed to mine") += mined;
    thread::sleep(Duration::from_millis(3000..7000));
  });
}
```

■ NO es busy wait. `expect()` sobre `write()` es bloqueante (espera pasiva). Si el lock no está disponible, el thread se suspende. Además hay sleep largo después del trabajo útil.

Ejemplo 2 — Filósofo con `try_acquire`

```
loop {
  let first_access = first_chopstick.acquire();
  match second_chopstick.try_acquire() {
    Ok(second) => { eat(); break; }
    Err(_) => { drop(first_access); } // ← libera y reintenta YA
  }
}
```

■ Sí es busy wait. `try_acquire` no bloquea. Al fallar, libera el primer palillo y reintenta inmediatamente toda la secuencia → patrón de reintento activo.

Ejemplo 3 — Consumidor con `loop + value.is_some()`

```
let t2 = thread::spawn(move || {
  loop {
    let lock = c2.lock().unwrap();
    if lock.value.is_some() {
      println!("{}", lock.value.unwrap());
      break;
    }
  }
});
```

■ Sí es busy wait. El loop verifica continuamente si `value` es `Some`. No usa `Condvar`. Solución: usar `Arc<(Mutex, Condvar)>` y `wait_while`.

Ejemplo 4 — `TcpStream::connect` en loop

```
loop {
  match TcpStream::connect("127.0.0.1:8080") {
    Ok(mut stream) => stream.write_all(message).expect("error"),
    Err(_) => {
      thread::sleep(Duration::from_millis(rand_short));
    }
  }
}
```

■ Sí es busy wait. Reintento activo de conexión. El `sleep` es corto y aleatorio, sigue gastando recursos en gestión de reintento.

4. Corrección de código — errores típicos

Error #1: Mezclar Actores con Condvar/Mutex/Semáforos

■ Es el error más frecuente en parciales 1c2025.

Los actores corren sobre un Arbiter (event loop async). Si el handler hace `cvar.wait()` o `sem.acquire()`, **bloquea todo el Arbiter** y otros actores no pueden procesar mensajes.

Síntomas del código mal hecho

- Struct de actor que tiene Condvar, Semaphore, o Mutex<Vec<T>> con espera bloqueante.
- Handler que llama a `cvar.wait_while()` o `sem.acquire()`.
- Handler que hace `await` sobre operaciones bloqueantes.

Cómo arreglarlo

- **Estado interno del actor:** guardar bool/Option para el recurso (`disponible: bool`, `ocupado_por: Option<Addr>`).
- **Cola interna:** guardar mensajes pendientes en un `Vec<MensajeEnEspera>`.
- **Notificación con `do_send`:** cuando el recurso se libera, enviar mensaje al actor en espera.
- **Lógica inversa:** si el actor no está listo, encolar el mensaje en lugar de re-enviarlo en bucle.

Patrón correcto de actor controlador de recurso

```
struct Recurso {
    disponible: bool,
    cola_espera: Vec<Addr<Cliente>>,
}

impl Handler<Solicitar> for Recurso {
    fn handle(&mut self, msg: Solicitar, _ctx) -> Self::Result {
        if self.disponible {
            self.disponible = false;
            msg.cliente.do_send(Concedido);
        } else {
            self.cola_espera.push(msg.cliente); // encolar
        }
    }
}

impl Handler<Liberar> for Recurso {
    fn handle(&mut self, _msg: Liberar, _ctx) -> Self::Result {
        if let Some(siguiete) = self.cola_espera.pop() {
            siguiete.do_send(Concedido); // notificar al siguiente
        } else {
            self.disponible = true;
        }
    }
}
```

Error #2: drop(guard) faltante

En Rust, los locks se liberan al salir de scope (RAII). Si el scope es largo, el lock queda tomado más tiempo del necesario, pudiendo causar deadlock o reducir paralelismo.

```
// MAL: el lock vive hasta el fin del bloque
let mut buf = buffer_local.lock().unwrap();
if buf.len() < N {
    buf.push(rand());
} else {
    // buf NO se libera antes del sleep → otros threads bloqueados
```



```

        thread::sleep(Duration::from_secs(1));
    }

    // BIEN: drop explícito antes del sleep
    let mut buf = buffer_local.lock().unwrap();
    if buf.len() < N {
        buf.push(rand());
    } else {
        drop(buf); // libera ANTES del sleep
        thread::sleep(Duration::from_secs(1));
    }
}

```

Error #3: Productor-Consumidor sin Condvar

Si el productor avisa al consumidor con un Semáforo y guarda los datos en un `Mutex<Option<T>>`, hay sobreescritura: si el productor produce 2 veces antes de que el consumidor lea, se pierde el primer dato.

■ Solución: usar canales mpsc o `Arc<(Mutex, Condvar)>`

```

// MAL: solo Semáforos + Mutex<Option>
let data = Arc::new(Mutex::new(None));
let producers_waiting = Arc::new(Semaphore::new(0));
// ...
*data.lock().unwrap() = Some(id); // sobreescribe!

// BIEN: canal mpsc transfiere ownership
let (tx, rx) = mpsc::channel();
tx.send(id).unwrap(); // productor
let id = rx.recv().unwrap(); // consumidor

```

Error #4: await dentro de handler de actor (cuidado)

Si un handler hace `.send(msg).await` sobre otro actor que tarda, **bloquea su Arbiter** hasta que el otro responda. Otros mensajes en su mailbox quedan sin procesar.

■ Solución: usar `do_send` en lugar de `send().await`, o rediseñar con notificación inversa.

```

// MAL: el handler espera bloqueando el arbiter
fn handle(&mut self, _msg, _ctx) {
    loop {
        for maquina in self.maquinas.iter() {
            match maquina.send(EstasLibre).await {
                Ok(true) => { maquina.do_send(Preparar); return; }
                Ok(false) => continue,
            }
        }
    }
}

// BIEN: actor Gerente notifica con do_send cuando hay máquina libre
struct Gerente {
    maquinas_libres: Vec<Addr<Maquina>>,
    baristas_esperando: Vec<Addr<Barista>>,
}

// Cuando una máquina se libera:
// gerente.do_send(MaquinaLibre(addr))
// El gerente notifica al primer barista en espera

```

5. Modelar con Actores — receta + ejemplos

Receta de 4 pasos

- **1. Identificar entidades reales** del enunciado: cada sustantivo importante = candidato a actor (Cliente, Mozo, Cocinero, Tanque, Tarjeta, etc.).
- **2. Para cada actor, definir el struct** con su estado interno: qué necesita recordar entre mensajes.
- **3. Para cada actor, listar los mensajes** que recibe: los verbos del enunciado (Solicitar, Consumir, Recargar, Liberar).
- **4. Identificar el recurso compartido limitado**: ese suele ser el actor cuello de botella que serializa con su mailbox.

Plantilla de respuesta

```
# Actor [NOMBRE]
- Estado interno: campo1: Tipo, campo2: Tipo
- Mensajes que recibe:
  • Mensaje1(payload): qué hace al recibirlo
  • Mensaje2(payload, AddrDestino): qué hace, a quién responde
- Notas: cómo asegura exclusión, si tiene cola de espera, etc.
```

Pistas para detectar el recurso compartido

- "*Solo N a la vez*" → recurso con capacidad limitada.
- "*Mientras nadie esté usando*" → exclusión mutua.
- "*Uno por vez*" → mailbox del actor serializa naturalmente.
- "*Compartido entre todos*" → actor central con estado.

Ejemplo 1: Estación de servicio (1c2025)

Enunciado: 3 tanques de nafta, 10 surtidores. Surtidores atienden a autos. Camión cisterna recarga, pero ningún tanque debe estar en uso mientras se recarga.

■ **Recurso compartido: cada tanque (acceso por surtidor o camión, exclusivo).**

```
# Actor Tanque (x3)
- Estado: nivel: f64, ocupado_por_surtidor: Option<Addr<Surtidor>>,
  esta_en_recarga: bool
- Mensajes:
  • SolicitarUso(litros, SurtidorAddr):
    Si ocupado_por es None y !esta_en_recarga:
      marca ocupación, descuenta nivel
      responde UsoAceptado al Surtidor
    Si no: responde Ocupado o encola
  • Liberar(): pone ocupado_por = None
  • Recargar(cantidad, CamionAddr):
    Si ocupado_por es None: marca esta_en_recarga = true,
      actualiza nivel, responde RecargaOk al Camion
    Si está ocupado: responde TanqueOcupado

# Actor Surtidor (x10)
- Estado: tanque_asignado: Addr<Tanque>
- Mensajes:
  • CargarAuto(litros): envía SolicitarUso al Tanque
  • UsoAceptado: despacha combustible, luego envía Liberar

# Actor CamionCisterna
- Estado: tanques_pendientes: Vec<Addr<Tanque>>
```

- Mensajes:
 - IniciarRecarga(tanque_addr): envía Recargar al Tanque
 - TanqueOcupado: reintenta más tarde (con timer)

Actor Auto

- Estado: tanque_propio_vacio: bool
- Mensajes:
 - CargarNafta: envía CargarAuto al Surtidor
 - TanqueLleno: termina

Ejemplo 2: Salón de videojuegos con tarjetas (1c2025-recu2)

Enunciado: Tarjetas recargables. En cumpleaños, los invitados comparten un **pool de dinero**: cada tarjeta descuenta del pool compartido.

■ **Recurso compartido: el pool de dinero (estado central, exclusivo).**

Actor PoolDinero

- Estado: saldo: u32
- Mensajes:
 - Gastar(monto, TarjetaAddr):
 - Si saldo >= monto: descuenta, responde GastoExitoso
 - Si no: responde SaldoInsuficiente
 - Recargar(monto): saldo += monto

Actor Tarjeta

- Estado: pool_addr: Addr<PoolDinero>, costo_juego: u32
- Mensajes:
 - Jugar: envía Gastar al PoolDinero
 - RespuestaGasto(exito): permite o deniega el juego al invitado

Actor VideoJuego

- Estado: costo: u32
- Mensajes:
 - IniciarJuego(TarjetaAddr): delega el gasto en la Tarjeta

Ejemplo 3: Restaurante con depósito (2c2024)

Enunciado: Mozos toman pedidos, los notifican a la cocina. Cocineros entran al depósito **uno por vez** a buscar ingredientes.

■ **Recurso compartido: depósito (1 cocinero a la vez).**

Actor Mozo

- Estado: pedidos_tomados: Map<MesaID, Pedido>, disponible: bool
- Mensajes:
 - TomarPedido(Pedido, ClienteAddr): registra y notifica a Cocina
 - PedidoListo(Comida): lleva al Cliente
 - Cobrar(Monto): procesa pago

Actor Deposito

- Estado: ocupado: bool, cola_espera: Vec<Addr<Cocinero>>
- Mensajes:
 - SolicitarAcceso(CocineroAddr):
 - Si !ocupado: marca ocupado, responde AccesoConcedido
 - Si ocupado: encola al cocinero
 - Liberar():
 - Si cola no vacía: saca siguiente y le da acceso
 - Si vacía: ocupado = false

Actor Cocinero

- Estado: pedido_actual: Option<Pedido>, deposito: Addr<Deposito>
- Mensajes:

- Preparar(Pedido): envía SolicitarAcceso al Deposito
- AccesoConcedido: busca ingredientes, prepara,
envía Liberar al Deposito,
envía PedidoListo al Mozo

Actor Cocina

- Estado: cola_pedidos: Vec<Pedido>
- Mensajes:
 - NotificarPedido(Pedido, MozoAddr): encola, despacha a Cocinero libre

Ejemplo 4: Cafetería con baristas (2c2024-recu1)

Enunciado: Clientes piden café. Baristas usan **2 máquinas de café** (solo un barista por máquina a la vez).

Actor Cajero

- Estado: cola_pedidos: Vec<Pedido>, baristas: Vec<Addr<Barista>>
- Mensajes:
 - NuevoPedido(Pedido, ClienteAddr): encola y notifica a baristas

Actor Barista

- Estado: pedido_actual: Option<Pedido>, maquinas: Vec<Addr<Maquina>>
- Mensajes:
 - TomarTrabajo: agarra pedido, envía SolicitarUso a una Máquina
 - UsoAceptado(MaquinaID): prepara, envía PedidoListo al Cliente
 - Ocupada: intenta con otra máquina o espera notificación

Actor MaquinaCafe (x2)

- Estado: ocupado_por: Option<Addr<Barista>>, cola: Vec<Addr<Barista>>
- Mensajes:
 - SolicitarUso(BaristaAddr):
 - Si None: marca ocupado, responde UsoAceptado
 - Si Some: encola al barista
 - Liberar(): pone None, notifica al primero en cola

Actor Cliente

- Estado: nombre: String, pedido: Pedido
- Mensajes:
 - PedidoListo(Comida): recibe el café

6. Redes de Petri — clásicos del parcial

Reglas de modelado

- **Lugar** (círculo) = estado o condición. **Token** (punto) = instancia presente.
- **Transición** (rectángulo) = evento. Se dispara si todos sus lugares de entrada tienen $\geq$ peso de arco.
- **Token inicial** en lugar = el proceso comienza en ese estado.
- **Lugar con N tokens** = capacidad del recurso (ej: tamaño del buffer).
- **Arco inhibidor** (círculo en el extremo) = la transición se habilita *solo si el lugar está vacío*.

Productor-Consumidor con buffer infinito

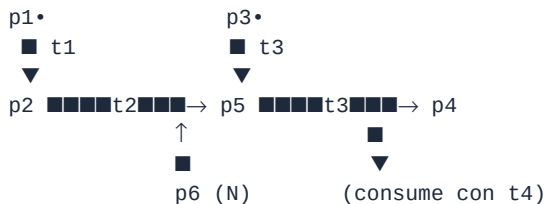
Estructura mínima: 4 lugares + 4 transiciones.

- **p1**: Productor listo (1 token inicial).
- **p2**: Productor terminó de producir, listo para depositar.
- **p3**: Consumidor listo (1 token inicial).
- **p4**: Consumidor terminó de tomar, listo para consumir.
- **p5**: Buffer (acumula tokens producidos).
- **t1**: producir **t2**: depositar ($p2 \rightarrow p5, p1$) **t3**: tomar ($p3, p5 \rightarrow p4$) **t4**: consumir.

Productor-Consumidor con buffer acotado (tamaño N)

Igual que el infinito, pero se agrega **p6** con N tokens iniciales (espacios libres).

- **Para depositar (t2)**: consume token de p6 (gasta un espacio libre) y agrega a p5.
- **Para tomar (t3)**: consume token de p5 (un dato) y agrega a p6 (libera espacio).



Lector-Escritor sin preferencia

Dos cohortes: N lectores y M escritores.

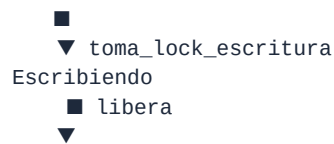
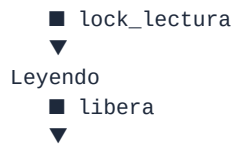
- **Lock de lectura**: habilitado si NO hay escritor escribiendo. Múltiples lectores pueden leer simultáneamente.
- **Lock de escritura**: habilitado si NO hay nadie leyendo NI escribiendo. Solo un escritor a la vez.
- **Sin preferencia**: si llegan lector y escritor a la vez, cualquiera puede ganar (no hay garantía).

Lector-Escritor con preferencia de escritura

Se agrega un lugar adicional **Escritores esperando** y un **arco inhibidor** desde ese lugar a la transición *lock_lectura*.

- Si hay un escritor esperando $\rightarrow$ la transición *lock_lectura* queda inhibida $\rightarrow$ nuevos lectores no entran.
- Cuando los lectores actuales terminan, el escritor toma el lock.
- Esto resuelve la **starvation de escritores** pero puede causar starvation de lectores.





Mutex / Sección Crítica

Un lugar Mutex con 1 token inicial. Cada proceso que quiere entrar consume el token; al salir lo devuelve.

- Si N procesos compiten por la sección crítica, todos comparten el mismo lugar mutex.

Banquero (con barreras)

Modelo común: cada inversor pasa por fases de `inicio_semana` → `invertir` → `fin_semana`, sincronizadas con barreras.

- **Barrera de inicio:** transición que requiere 1 token de cada inversor para dispararse.
- **Barrera de fin:** ídem para sincronizar al cierre de la semana.

7. Chuleta — todo en una página

Decisión de modelo

Pista	Modelo
CPU-bound, datos independientes	Fork-Join
I/O-bound, red, APIs, archivos	Async
Estado reactivo por entidad	Actores
Estado compartido modificado	Mutex/RwLock
Recurso limitado a N usos	Semáforo

V/F — los siempre-FALSO

- `poll()` es bloqueante → F (nunca bloquea)
- `async fn` ejecuta al llamarla → F (devuelve Future)
- actor protege estado con mutex → F (es privado por aislamiento)
- fork-join tiene races → F (subtareas independientes)
- `condvar` solo despierta con signal → F (spurious wakeups)
- threads tienen memoria separada → F (solo procesos)
- tasks tienen stack → F (máquina de estados)

Busy wait — decisión

- loop con `lock()/acquire()/wait()/recv()` → NO busy wait
- loop con `try_lock()/try_acquire()/is_some()` → Sí busy wait
- loop con sleep largo + trabajo útil → NO busy wait
- loop con sleep corto + chequeo → Sí busy wait disimulado
- `TcpStream::connect` en loop → Sí busy wait

Patrones de diseño con Actores

- **Actor controlador de recurso:** estado disponible: `bool` + cola interna `Vec<Addr>`.
- **Notificación inversa:** usar `do_send` cuando se libera, NO bloquear.
- **Mailbox = lock implícito:** mensajes serializados naturalmente.
- **NUNCA** mezclar actores con `Condvar/Mutex/Semaphore` bloqueante.

Errores comunes a evitar

- Usar `Condvar` dentro de actor → bloquea el Arbiter.
- Olvidar `drop(guard)` en scopes largos → bloquea otros threads.
- Productor-Consumidor con `Mutex` → sobrescribe datos.
- `if` en lugar de `while` con `condvar` → bug por spurious wakeup.
- `await` en handler → bloquea el arbiter completo.

Plantillas para responder rápido

■ Modelo de concurrencia

[MODELO]. [TAREA] es [CPU/I-O/reactiva]. Este modelo es ideal porque [VENTAJA]. Permite [BENEFICIO]. [OTRO MODELO] no sirve porque [POR QUÉ].

■ Diseño con Actores

```
# Actor [Nombre]
- Estado: campos
- Mensajes:
  • Mensaje1(payload): efecto
- Notas: cómo asegura exclusión, cola de espera, recurso limitado
```

■ Justificar busy wait

[SI/NO] es busy wait. El thread [está atrapado en un loop verificando una condición rápida sin ceder CPU / usa primitivas bloqueantes que lo suspenden]. [Mencionar try_acquire / sleep corto / falta de Condvar].